

AiSD Lista 3

Dominik Kowalczyk

16 stycznia 2025

1 Algorytm CUT ROD rozwiązujący problem podziału pręta

Chcemy znaleźć maksymalny zysk po jakim możemy sprzedać pręt długości n , mając możliwość dzielić go na kawałki. W celu rozwiązania tego problemu optymalizacyjnego może pomóc nam algorytm rozcinań pręta - CUT ROD, które przedstawimy implementację naiwną, ze spamiętywaniem oraz iteracyjną.

1.1 Naiwny

Listing 1: Kod funkcji rozcinań pręta w wersji naiwnej

```
1 float CUTROD_NAIWNY(float* p, int n) {  
2     if (n == 0) {  
3         return 0;  
4     }  
5     float q = inf;  
6     for (int i = 1; i <= n; ++i) {  
7         q = max(q, p[i - 1] +  
8             CUTROD_NAIWNY(p, n - i));  
9     }  
10    return q;  
11 }
```

W naszym algorytmie funkcja *CUTROD* przyjmuje argumenty p , który jest wskaźnikiem przechowującym ceny różnych długości pręta oraz n reprezentująca długość pręta.

Na początku wchodzimy do pętli *for* sprawdzająca, czy nasz pręt nie jest długości 0 (jeśli jest zwracamy 0). Następnie tworzy-

my zmienną q przechowującą maksymalny zysk i na start ustawiamy ją na $-\infty$. Następnie przechodzimy do pętli *for* przechodzącej po wszystkich możliwych podziałach pręta. Następnie obliczamy dla każdego podziału maksymalny zysk poprzez wybór za pomocą funkcji *max* zysk aktualnie zapisany pod zmienną q lub ceną odcinka długości i lub pozostałej części pręta $n - i$ poprzez rekurencyjne wywołanie *CUTROD NAIWNY*. Na sam koniec zwracamy q .

1.2 Ze spamiętywaniem

Listing 2: Kod funkcji rozcinania pręta w wersji ze spamiętywaniem

```

1  float MEMORIZED_CUT_ROD(float* p, float* r,
2  int* s, int n) {
3      if (r[n] >= 0) {
4          return r[n];
5      }
6      float q;
7      if (n == 0) {
8          q = 0;
9      } else {
10         q = inf;
11
12         for (int i = 1; i <= n; ++i) {
13
14             float m = p[i - 1] +
15                 MEMORIZED_CUT_ROD(p,
16                 r, s, n - i);
17
18             if (m > q) {
19                 q = m;
20                 s[n] = i;
21             }
22         }
23     }
24     r[n] = q;
25     return q;
26 }
```

W przypadku algorytmu ze spamiętywaniem mamy do czynienia

z większą ilością argumentów. Oprócz p i n mamy wskaźnik (do tablicy) r gdzie przechowywane jest maksymalny możliwy zysk dla danej długości pręta, jak również wskaźnik (do tablicy) s , który przechowuje informacje o optymalnych cięciach pręta.

Mamy warunki początkowe, które działają na tej samej zasadzie co w algorytmie naiwnym. Znacząca zmiana zachodzi w pętli *for* gdzie tworzymy zmienną tymczasową m wyliczając dla każdego cięcia i zysk. Następnie mamy krótką pętlę *if*, która przy warunku, gdy nasz zysk wyliczony i przechowywany w zmiennej tymczasowej m jest większy od q (zysku maksymalnego) to staje się nim, a nasza tablica optymalnego cięcia zapamiętuje wartość i . Ponownie zwracamy nasz zysk maksymalny.

1.3 iteracyjna

Listing 3: Kod funkcji rozcinania pręta w wersji ze spamiętywaniem

```

1 float EXT_BOT_UP_CUT_ROD(float* p, float* r,
2   int* s, int n) {
3     r[0] = 0;
4     for (int j = 1; j <= n; ++j) {
5         float q = inf;
6
7         for (int i = 1; i <= j; ++i) {
8             if (q < p[i - 1] +
9                 r[j - i]) {
10
11                 q = p[i - 1] +
12                     r[j - i];
13
14                 s[j] = i;
15             }
16         }
17         r[j] = q;
18     }
19     return r[n];
20 }
```

W wersji iteracyjnej mamy takie same argumenty jak w przypadku ze spamiętywaniem: p , r , s , n . Początkowo ustawiamy mak-

symalny zysk na 0 ($r[0] = 0$). Wchodzimy do pętli *for*, ustawiamy ponownie q na $-\infty$. Wchodzimy do wewnętrznej pętli *for*, gdzie iterujemy przez wszystkie możliwe cięcia pręta.

1. Dla każdego cięcia obliczana jest suma:

$$\text{zysk} = p[i - 1] + r[j - i],$$

gdzie:

- $p[i - 1]$ - cena odciętego kawałka pręta o długości i
 - $r[j - i]$ - maksymalny zysk dla pozostałej długości pręta $j - i$, (który już wcześniej został obliczony).
2. Jeżeli obliczony zysk jest większy niż dotychczasowa maksymalna wartość q , następuje aktualizacja:
 - q przyjmuje nową, większą wartość zysku,
 - $s[j]$ zapisuje długość cięcia i jako optymalne pierwsze cięcie dla pręta o długości j .

Pętla kończy się, gdy wszystkie możliwe długości cięć zostaną przeanalizowane, a zmienna q zawiera maksymalny możliwy zysk dla pręta o długości j .

2 LCS - problem znajdowania najdłuższego wspólnego podciągu

Przy analizie algorytmu rozwiązującego problem znajdowania najdłuższego wspólnego podciągu przyjrzymy się implementacji rekurencyjnej ze spamiętywaniem oraz wersji iteracyjnej.

2.1 rekurencyjna ze spamiętywaniem

Listing 4: Kod funkcji znajdowania nadłuższego wspólnego podciągu w wersji rekurencyjnej

```

1 int LCS_R(const string &w1, const string &w2,
2     int m, int n, int** s) {
3     if (s[m][n] != -1) {
4         return s[m][n];
    
```

```

5      }
6      if (m == 0 || n == 0) {
7          return s[m][n] = 0;
8      }
9      if (w1[m - 1] == w2[n - 1]) {
10         return s[m][n] = 1 + LCS_R(w1,
11                                     w2, m - 1, n - 1, s);
12     }
13     return s[m][n] = max(LCS_R(w1, w2, m,
14                                 n - 1, s), LCS_R(w1, w2, m - 1, n, s));
15 }

```

Argumentami funkcji w implementacji rekurencyjnej są jakieś litery oraz ich długości $w1$, $w2$ i odpowiednio m oraz n , a także tablica $s[]$ mająca za zadanie przechowywać maksymalne długości podciągów. Jak w każdej rekurencyjnej funkcji mamy warunki początkowe. Najważniejszą częścią tej implementacji jest podzielenie sytuacji na dwa przypadki - w której mamy styczność z takimi samymi ostatnimi znakami wyrazu (dodajemy 1 i wywołujemy rekurencyjne dla pozostałych) lub są różne. W takim przypadku obliczamy dwie możliwości (z pominięciem ostatniego znaku $w1$ lub $w2$) i zwracamy wartość maksymalną. Tablica $s[]$ jest zwracana i to ona odpowiada za funkcję spamiętywania.

2.2 iteracyjna

Listing 5: Kod funkcji znajdowania nadłuższego wspólnego podciągu w wersji iteracyjnej

```

1  char** LCS_I(const string &w1, const
2      string &w2, int& dlugosc) {
3      int m = w1.length();
4      int n = w2.length();
5
6      int** c = new int*[m + 1];
7      char** b = new char*[m + 1];
8      for (int i = 0; i <= m; i++) {
9          c[i] = new int[n + 1];
10         b[i] = new char[n + 1];
11     }

```

```

12
13         for (int i = 0; i <= m; i++) {
14             c[i][0] = 0;
15         }
16         for (int j = 0; j <= n; j++) {
17             c[0][j] = 0;
18         }

```

Przypadek iteracyjny został napisany na podstawie pseudokodu, więc nie jest on zbyt ciekawym kodem. W mojej implementacji mamy tylko trzy argumenty: $w1$, $w2$ oraz zmienna *dlugosc*. Dopiero wewnątrz funkcji definiuje zmienne m oraz n jak również tworzę dwie pomocnicze tablice: $b[]$ odpowiedzialna za przechowywanie kierunków do odtworzenia LCS oraz $c[]$ przechowująca długość ciągu.

Listing 6: Kod funkcji znajdowania nadłuższego wspólnego podciągu w wersji iteracyjnej

```

1  if (w1[i - 1] == w2[j - 1]) {
2      c[i][j] = c[i - 1][j - 1] + 1;
3      b[i][j] = '\\';
4  } else if (c[i - 1][j] >= c[i][j - 1]) {
5      c[i][j] = c[i - 1][j];
6      b[i][j] = '|';
7  } else {
8      c[i][j] = c[i][j - 1];
9      b[i][j] = '-';
10 }
11 }
12 dlugosc = c[m][n];

```

przeanalizujemy najciekawszy część tego algorytmu:

- Dla każdej pary (i, j) , gdzie $1 \leq i \leq m$ oraz $1 \leq j \leq n$:
 1. Jeśli znaki w obu ciągach są równe, tj. $w1[i-1] = w2[j-1]$, to:

$$c[i][j] = c[i-1][j-1] + 1$$
 oraz ustawiamy kierunek na ukośny
 2. Jeśli znaki są różne, porównujemy wartości:

$$c[i-1][j] \quad \text{oraz} \quad c[i][j-1]$$

– Jeśli $c[i-1][j] \geq c[i][j-1]$, to:

$$c[i][j] = c[i-1][j]$$

oraz ustawiamy kierunek pionowy

– W przeciwnym przypadku:

$$c[i][j] = c[i][j-1]$$

oraz ustawiamy kierunek poziomy

Na sam koniec przypisujemy do argumentu *dlugosc* wartość najdłuższego wspólnego podciągu. Funkcja zwraca $b[]$.